

Contents

DuckDB Backend	1
Introduction	1
Connecting with DuckDB	2
Building an Atlas on DuckDB	2
Standard Queries on DuckDB	3
DuckDB Analytical Queries	3
Window Functions: Ranking Edges Within Relation Types	3
Common Table Expressions: Transitive Influence	4
Aggregate Statistics by Source Document	5
Backend Comparison	6
When to Choose Each Backend	6
File-Based DuckDB	7
Summary	7

DuckDB Backend

- Introduction
- Connecting with DuckDB
- Building an Atlas on DuckDB
- Standard Queries on DuckDB
- DuckDB Analytical Queries
 - Window Functions: Ranking Edges Within Relation Types
 - Common Table Expressions: Transitive Influence
 - Aggregate Statistics by Source Document
- Backend Comparison
 - When to Choose Each Backend
- File-Based DuckDB
- Summary

Introduction

CSQL.jl supports multiple database backends via DBInterface.jl. While SQLite is the default (lightweight, zero-config), DuckDB offers advantages for analytical workloads:

- **Columnar storage** — faster aggregation and scanning
- **Vectorized execution** — efficient batch processing
- **Rich analytical SQL** — window functions, CTEs, advanced aggregates
- **Larger-than-memory** — efficient handling of large atlases

This vignette demonstrates using DuckDB as the CSQL backend and leveraging its analytical capabilities.

Connecting with DuckDB

```
using CSQL

# In-memory DuckDB database
csql_duck = connect_csql(; backend=:duckdb)
println("Connected to DuckDB backend")
println("Type: ", typeof(csql_duck.db))

Connected to DuckDB backend
Type: DuckDB.DB
```

Building an Atlas on DuckDB

The builder API is identical — just pass the DuckDB connection:

```
builder = AtlasBuilder()

# Climate-health causal network
triples = [
    ("Temperature increase", "increases", "Vector habitat range", "ipcc2023", 0.91),
    ("Vector habitat range", "increases", "Vector population", "who2024", 0.85),
    ("Vector population", "increases", "Disease transmission", "who2024", 0.88),
    ("Disease transmission", "increases", "Morbidity", "lancet2024", 0.82),
    ("Morbidity", "increases", "Healthcare burden", "lancet2024", 0.79),
    ("Temperature increase", "increases", "Drought frequency", "ipcc2023", 0.87),
    ("Drought frequency", "reduces", "Food security", "fao2024", 0.83),
    ("Food security", "reduces", "Nutritional status", "fao2024", 0.76),
    ("Nutritional status", "reduces", "Immune function", "nejm2024", 0.72),
    ("Immune function", "reduces", "Disease transmission", "nejm2024", 0.68),
    ("Urbanization", "increases", "Vector habitat range", "ecology2024", 0.74),
    ("Urbanization", "increases", "Contact rate", "ecology2024", 0.81),
    ("Contact rate", "increases", "Disease transmission", "who2024", 0.86),
    ("Healthcare investment", "increases", "Healthcare capacity", "who2024", 0.90),
    ("Healthcare capacity", "reduces", "Morbidity", "lancet2024", 0.84),
    ("Poverty", "reduces", "Healthcare access", "wb2024", 0.88),
    ("Healthcare access", "reduces", "Morbidity", "lancet2024", 0.80),
    ("Poverty", "increases", "Malnutrition", "wb2024", 0.85),
    ("Malnutrition", "reduces", "Immune function", "nejm2024", 0.78),
    ("Air pollution", "increases", "Respiratory disease", "lancet2024", 0.86),
    ("Temperature increase", "increases", "Air pollution", "ipcc2023", 0.73),
]

for (s, r, o, doc, score) in triples
    add_triple!(builder, s, r, o; doc_id=doc, score=score)
end
```

```
build!(builder, csql_duck.db)
```

```
stats = statistics(csql_duck)
println("Atlas: $(stats[:n_nodes]) nodes, $(stats[:n_edges]) edges")
Atlas: 19 nodes, 21 edges
```

Standard Queries on DuckDB

All CSQL.jl queries work identically:

```
println("=== Top 5 Backbone ===")
for r in backbone(csql_duck; limit=5)
    println(" $(r.src) → $(r.dst) score=$(round(r.score_sum; digits=2))")
end

println("\n=== Causal Hubs ===")
for h in causal_hubs(csql_duck; limit=5)
    println(" $(h.concept): mass=$(round(h.out_mass; digits=2)), edges=$(h.n_edges)")
end

=== Top 5 Backbone ===
temperature increase → vector habitat range score=0.91
healthcare investment → healthcare capacity score=0.9
vector population → disease transmission score=0.88
poverty → healthcare access score=0.88
temperature increase → drought frequency score=0.87

=== Causal Hubs ===
temperature increase: mass=2.51, edges=3
poverty: mass=1.73, edges=2
urbanization: mass=1.55, edges=2
healthcare investment: mass=0.9, edges=1
vector population: mass=0.88, edges=1
```

DuckDB Analytical Queries

DuckDB's analytical SQL capabilities enable sophisticated causal analysis:

Window Functions: Ranking Edges Within Relation Types

```
ranked = custom_query(csql_duck, """
    SELECT rel_type, src, dst, score_sum,
           rank_in_type
    FROM (
        SELECT e.rel_type,
               n1.label_canon AS src,
               n2.label_canon AS dst,
```

```

        e.score_sum,
        ROW_NUMBER() OVER (PARTITION BY e.rel_type ORDER BY e.score_sum DESC) AS rank_in_type
    FROM atlas_edges e
    JOIN atlas_nodes n1 ON e.src_id = n1.node_id
    JOIN atlas_nodes n2 ON e.dst_id = n2.node_id
) sub
WHERE rank_in_type <= 3
ORDER BY rel_type, rank_in_type
""")
println("Top 3 edges per relation type:")
current_type = ""
for r in ranked
    if r.rel_type != current_type
        current_type = r.rel_type
        println("\n $(current_type):")
    end
    println("    #$(r.rank_in_type) $(r.src) → $(r.dst) ($(round(r.score_sum; digits=2)))")
end

```

Top 3 edges per relation type:

INCREASES:

- #1 temperature increase → vector habitat range (0.91)
- #2 healthcare investment → healthcare capacity (0.9)
- #3 vector population → disease transmission (0.88)

REDUCES:

- #1 poverty → healthcare access (0.88)
- #2 healthcare capacity → morbidity (0.84)
- #3 drought frequency → food security (0.83)

Common Table Expressions: Transitive Influence

```

influence = custom_query(csql_duck, """
WITH direct AS (
    SELECT n1.label_canon AS concept,
           SUM(e.score_sum) AS direct_influence
    FROM atlas_edges e
    JOIN atlas_nodes n1 ON e.src_id = n1.node_id
    GROUP BY n1.label_canon, n1.node_id
),
indirect AS (
    SELECT n1.label_canon AS concept,
           SUM(e1.score_sum * e2.score_sum) AS indirect_influence
    FROM atlas_edges e1
    JOIN atlas_edges e2 ON e1.dst_id = e2.src_id

```

```

        JOIN atlas_nodes n1 ON e1.src_id = n1.node_id
        GROUP BY n1.label_canon, n1.node_id
    )
    SELECT d.concept,
           ROUND(d.direct_influence, 2) AS direct,
           ROUND(COALESCE(i.indirect_influence, 0), 2) AS indirect,
           ROUND(d.direct_influence + COALESCE(i.indirect_influence, 0), 2) AS total
    FROM direct d
    LEFT JOIN indirect i ON d.concept = i.concept
    ORDER BY total DESC
    LIMIT 10
    """)
println("\nDirect vs indirect influence:")
for r in influence
    println("  $(r.concept): direct=$(r.direct), indirect=$(r.indirect), total=$(r.total)")
end

```

Direct vs indirect influence:

```

temperature increase: direct=2.51, indirect=2.12, total=4.63
poverty: direct=1.73, indirect=1.37, total=3.1
urbanization: direct=1.55, indirect=1.33, total=2.88
healthcare investment: direct=0.9, indirect=0.76, total=1.66
vector population: direct=0.88, indirect=0.72, total=1.6
vector habitat range: direct=0.85, indirect=0.75, total=1.6
contact rate: direct=0.86, indirect=0.71, total=1.57
healthcare capacity: direct=0.84, indirect=0.66, total=1.5
disease transmission: direct=0.82, indirect=0.65, total=1.47
drought frequency: direct=0.83, indirect=0.63, total=1.46

```

Aggregate Statistics by Source Document

```

doc_stats = custom_query(csql_duck, """
    SELECT es.doc_id,
           COUNT(DISTINCT es.edge_id) AS n_edges,
           COUNT(DISTINCT e.src_id) AS n_src_nodes,
           COUNT(DISTINCT e.dst_id) AS n_dst_nodes,
           ROUND(AVG(es.score), 3) AS avg_score,
           ROUND(STDDEV(es.score), 3) AS std_score
    FROM atlas_edge_support es
    JOIN atlas_edges e ON es.edge_id = e.edge_id
    GROUP BY es.doc_id
    ORDER BY n_edges DESC
    """)
println("\nDocument contribution statistics:")
for d in doc_stats
    std = d.std_score === missing ? 0.0 : d.std_score
end

```

```

        println(" $(d.doc_id): $(d.n_edges) edges, $(d.n_src_nodes) sources, $(d.n_dst_nodes) targets")
    end
end

```

Document contribution statistics:

```

lancet2024: 5 edges, 5 sources, 3 targets, avg=0.822 ± 0.029
who2024: 4 edges, 4 sources, 3 targets, avg=0.873 ± 0.022
ipcc2023: 3 edges, 1 sources, 3 targets, avg=0.837 ± 0.095
nejm2024: 3 edges, 3 sources, 2 targets, avg=0.727 ± 0.05
fao2024: 2 edges, 2 sources, 2 targets, avg=0.795 ± 0.049
ecology2024: 2 edges, 1 sources, 2 targets, avg=0.775 ± 0.049
wb2024: 2 edges, 1 sources, 2 targets, avg=0.865 ± 0.021

```

Backend Comparison

Both backends produce identical results — the choice depends on workload:

```

# Same atlas on SQLite
csql_lite = connect_csql(; backend=:sqlite)
builder2 = AtlasBuilder()
for (s, r, o, doc, score) in triples
    add_triple!(builder2, s, r, o; doc_id=doc, score=score)
end
build!(builder2, csql_lite.db)

# Compare results
bb_duck = backbone(csql_duck; limit=5)
bb_lite = backbone(csql_lite; limit=5)

println("Backend comparison (top 5 backbone):")
println("  DuckDB                                | SQLite")
println("  " * "-"^35 * "|" * "-"^35)
for i in 1:min(5, length(bb_duck), length(bb_lite))
    d = bb_duck[i]
    l = bb_lite[i]
    println(" $(rpad(d.src * " → " * d.dst, 35)) | $(l.src) → $(l.dst)")
end

```

Backend comparison (top 5 backbone):

```

DuckDB                                | SQLite
-----|-----
temperature increase → vector habitat range | temperature increase → vector habitat range
healthcare investment → healthcare capacity | healthcare investment → healthcare capacity
vector population → disease transmission | vector population → disease transmission
poverty → healthcare access                | poverty → healthcare access
temperature increase → drought frequency | temperature increase → drought frequency

```

When to Choose Each Backend

Feature	SQLite	DuckDB
Setup	Zero config	Zero config
Small atlases (< 10K edges)	Fast	Fast
Large atlases (> 100K edges)	Slower	Columnar scan
Window functions	Limited	Full support
STDDEV, advanced aggregates		
File persistence	<code>.sqlite</code>	<code>.duckdb</code>
Concurrent reads	Limited	

File-Based DuckDB

For persistent storage:

```
# Would create a file on disk:
# csql_persistent = connect_csql(; backend=:duckdb, path="my_atlas.duckdb")
# build!(builder, csql_persistent.db)
println("Use connect_csql(; backend=:duckdb, path=\"atlas.duckdb\") for persistent storage")
Use connect_csql(; backend=:duckdb, path="atlas.duckdb") for persistent storage
```

Summary

This vignette demonstrated:

- **connect_csql(; backend=:duckdb)** — using DuckDB as the CSQL backend
- **Identical API** — all CSQL.jl functions work with both backends
- **DuckDB analytics** — window functions, CTEs, STDDEV, advanced aggregates
- **Backend selection** — guidelines for choosing SQLite vs DuckDB